

LAB ASSIGNMENT – 16.1

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

- Tables: Students, Courses, Enrollments.
- Define primary keys, foreign keys, and relationships.
- Generate queries:

o Insert a new student record. o Fetch all courses enrolled by a student. o Count number of students in each course.

Expected Output:

- SQL CREATE TABLE statements and queries for student– course relationships.

PROMPT:

Generate SQL schema for Students, Courses, Enrollments with PK, FK and queries for insert, fetch courses by student, and count students per course import sqlite3

CODE:

```
conn = sqlite3.connect("student.db") cur
= conn.cursor()
```

```
# Create Tables cur.execute("CREATE TABLE IF NOT EXISTS Students (id INTEGER PRIMARY KEY,
name TEXT)") cur.execute("CREATE TABLE IF NOT EXISTS Courses (id INTEGER PRIMARY KEY,
course_name TEXT)") cur.execute("""CREATE TABLE IF NOT EXISTS Enrollments (
    id INTEGER PRIMARY KEY,
```

```
        student_id INTEGER,
course_id INTEGER,
        FOREIGN KEY(student_id) REFERENCES Students(id),
        FOREIGN KEY(course_id) REFERENCES Courses(id))""")

# Insert student cur.execute("INSERT INTO Students (name)
VALUES ('Priya')")

# Fetch courses by student cur.execute("""SELECT
course_name FROM Courses
        JOIN Enrollments ON Courses.id = Enrollments.course_id
WHERE student_id = 1""") print(cur.fetchall())

# Count students per course cur.execute("""SELECT course_name,
COUNT(student_id) FROM Courses
        JOIN Enrollments ON Courses.id = Enrollments.course_id
GROUP BY course_id""") print(cur.fetchall())

conn.commit() conn.close()
```

```
1 # Generate SQL schema for Students, Courses, Enrollments with PK, FK and queries for insert, fetch courses by student, and
2 import sqlite3
3
4 conn = sqlite3.connect("student.db")
5 cur = conn.cursor()
6
7 # Create Tables
8 cur.execute("CREATE TABLE IF NOT EXISTS Students (id INTEGER PRIMARY KEY, name TEXT)")
9 cur.execute("CREATE TABLE IF NOT EXISTS Courses (id INTEGER PRIMARY KEY, course_name TEXT)")
10 cur.execute("""CREATE TABLE IF NOT EXISTS Enrollments (
11     id INTEGER PRIMARY KEY,
12     student_id INTEGER,
13     course_id INTEGER,
14     FOREIGN KEY(student_id) REFERENCES Students(id),
15     FOREIGN KEY(course_id) REFERENCES Courses(id)""")
16
17 # Insert student
18 cur.execute("INSERT INTO Students (name) VALUES ('Priya')")
19
20 # Fetch courses by student
21 cur.execute("""SELECT course_name FROM Courses
22     JOIN Enrollments ON Courses.id = Enrollments.course_id
23     WHERE student_id = 1""")
24 print(cur.fetchall())
25
26 # Count students per course
27 cur.execute("""SELECT course_name, COUNT(student_id) FROM Courses
28     JOIN Enrollments ON Courses.id = Enrollments.course_id
29     GROUP BY course_id""")
30 print(cur.fetchall())
31
32 conn.commit()
33 conn.close()
```

Justification (4 lines)

- Ensures proper relational mapping using foreign keys
- Avoids redundancy via normalized design
- Supports efficient joins for queries
- Aggregation helps in analytics like course popularity

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:

o List all appointments for a specific doctor. o Retrieve patient history by patient ID.

o Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.

PROMPT:

Design hospital DB with Doctors, Patients, Appointments with constraints and queries for doctor appointments, patient history, and patient count per doctor. \

CODE:

```
import sqlite3
```

```
conn = sqlite3.connect("hospital.db") cur
```

```
= conn.cursor()
```

```
cur.execute("CREATE TABLE Doctors (id INTEGER PRIMARY KEY, name TEXT)") cur.execute("CREATE
```

```
TABLE Patients (id INTEGER PRIMARY KEY, name TEXT)") cur.execute("""CREATE TABLE
```

```
Appointments (
```

```
    id INTEGER PRIMARY KEY,
```

```
    doctor_id INTEGER,        patient_id
```

```
    INTEGER,
```

```
    date TEXT,
```

```
    FOREIGN KEY(doctor_id) REFERENCES Doctors(id),
```

```
    FOREIGN KEY(patient_id) REFERENCES Patients(id))""")
```

```
# Queries cur.execute("SELECT * FROM Appointments WHERE
```

```
doctor_id = 1")
```

```
cur.execute("""SELECT * FROM Appointments
```

```
    WHERE patient_id = 1""")
```

```
cur.execute("""SELECT doctor_id, COUNT(patient_id)
```

```
    FROM Appointments GROUP BY doctor_id""")
```

```
print(cur.fetchall())
```

```
conn.commit() conn.close()
```

```

1 # Design hospital db with Doctors, Patients, Appointments with constraints and queries for doctor appointments, patient his
2 import sqlite3
3
4 conn = sqlite3.connect("hospital.db")
5 cur = conn.cursor()
6
7 cur.execute("CREATE TABLE Doctors (id INTEGER PRIMARY KEY, name TEXT)")
8 cur.execute("CREATE TABLE Patients (id INTEGER PRIMARY KEY, name TEXT)")
9 cur.execute("""CREATE TABLE Appointments (
10     id INTEGER PRIMARY KEY,
11     doctor_id INTEGER,
12     patient_id INTEGER,
13     date TEXT,
14     FOREIGN KEY(doctor_id) REFERENCES Doctors(id),
15     FOREIGN KEY(patient_id) REFERENCES Patients(id)""")
16
17 # Queries
18 cur.execute("SELECT * FROM Appointments WHERE doctor_id = 1")
19
20 cur.execute("""SELECT * FROM Appointments
21     WHERE patient_id = 1""")
22
23 cur.execute("""SELECT doctor_id, COUNT(patient_id)
24     FROM Appointments GROUP BY doctor_id""")
25
26 print(cur.fetchall())
27
28 conn.commit()
29 conn.close()

```

Justification

- Maintains integrity using foreign keys
- Tracks relationships between doctors & patients
- Enables history retrieval via joins
- Aggregation supports workload analysis

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:

o Retrieve all books currently issued. o Find
overdue books (loan date > 30 days). o Count
number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

PROMPT:

Create library DB schema with Books, Members, Loans and queries for issued books, overdue books, and count per member with indexing suggestions import sqlite3

CODE:

```
conn = sqlite3.connect("library.db") cur
```

```
= conn.cursor()
```

```
cur.execute("CREATE TABLE Books (id INTEGER PRIMARY KEY, title TEXT)") cur.execute("CREATE
```

```
TABLE Members (id INTEGER PRIMARY KEY, name TEXT)") cur.execute("""CREATE TABLE Loans
```

```
(
```

```
    id INTEGER PRIMARY KEY,
```

```
    book_id INTEGER,        member_id
```

```
    INTEGER,        loan_date TEXT,
```

```
    FOREIGN KEY(book_id) REFERENCES Books(id),
```

```
    FOREIGN KEY(member_id) REFERENCES Members(id))""")
```

```
# Issued books cur.execute("""SELECT
```

```
title FROM Books
```

```
    JOIN Loans ON Books.id = Loans.book_id""")
```

```
# Overdue books (>30 days simulated)
```

```
cur.execute("""SELECT title FROM Books
```

```
    JOIN Loans ON Books.id = Loans.book_id
```

```
    WHERE julianday('now') - julianday(loan_date) > 30""")
```

```
conn.commit() conn.close()
```

```

1 # Create library DB schema with Books, Members, and queries for issued books, overdue books, and count per member with
2 import sqlite3
3
4 conn = sqlite3.connect("library.db")
5 cur = conn.cursor()
6
7 cur.execute("CREATE TABLE Books (id INTEGER PRIMARY KEY, title TEXT)")
8 cur.execute("CREATE TABLE Members (id INTEGER PRIMARY KEY, name TEXT)")
9 cur.execute("""CREATE TABLE Loans (
10 |         |         |         |         |
11 |         |         |         |         |         id INTEGER PRIMARY KEY,
12 |         |         |         |         |         book_id INTEGER,
13 |         |         |         |         |         member_id INTEGER,
14 |         |         |         |         |         loan_date TEXT,
15 |         |         |         |         |         FOREIGN KEY(book_id) REFERENCES Books(id),
16 |         |         |         |         |         FOREIGN KEY(member_id) REFERENCES Members(id))""")
17
18 # Issued books
19 cur.execute("""SELECT title FROM Books
20 |         |         |         |         |
21 |         |         |         |         |         JOIN Loans ON Books.id = Loans.book_id""")
22
23 # Overdue books (>30 days simulated)
24 cur.execute("""SELECT title FROM Books
25 |         |         |         |         |
26 |         |         |         |         |         JOIN Loans ON Books.id = Loans.book_id
27 |         |         |         |         |         WHERE julianday('now') - julianday(loan_date) > 30""")
28
29 # Count per member
30 cur.execute("""SELECT member_id, COUNT(book_id)
31 |         |         |         |         |
32 |         |         |         |         |         FROM Loans GROUP BY member_id""")
33
34 print(cur.fetchall())
35
36 conn.commit()
37 conn.close()

```

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:

o Retrieve all orders by a user.

o Find the most purchased product. o Calculate total revenue in a given month.

- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

PROMPT:

Design e-commerce DB schema and queries for user orders, most purchased product, and monthly revenue with normalization and optimization import sqlite3

CODE:

```
conn = sqlite3.connect("ecommerce.db") cur
= conn.cursor()
```

```
cur.execute("CREATE TABLE Users (id INTEGER PRIMARY KEY, name TEXT)") cur.execute("CREATE
TABLE Products (id INTEGER PRIMARY KEY, name TEXT, price REAL)") cur.execute("CREATE
TABLE Orders (id INTEGER PRIMARY KEY, user_id INTEGER, date TEXT)") cur.execute("""CREATE
TABLE OrderDetails (
    id INTEGER PRIMARY KEY,
    order_id INTEGER,
product_id INTEGER,    quantity
INTEGER,
    FOREIGN KEY(order_id) REFERENCES Orders(id),
    FOREIGN KEY(product_id) REFERENCES Products(id))""")
```



```
# Orders by user cur.execute("SELECT * FROM Orders  
WHERE user_id = 1")
```

```
# Most purchased product cur.execute("""SELECT  
product_id, SUM(quantity) as total  
FROM OrderDetails GROUP BY product_id  
ORDER BY total DESC LIMIT 1""")
```

```
# Revenue in month cur.execute("""SELECT SUM(Products.price *  
OrderDetails.quantity)  
FROM OrderDetails  
JOIN Products ON Products.id = OrderDetails.product_id""")
```

```
print(cur.fetchall())
```

```
conn.commit() conn.close()
```

```

1 # Design e-commerce DB schema and queries for user orders, most purchased product, and monthly revenue with normalization
2 import sqlite3
3
4 conn = sqlite3.connect("ecommerce.db")
5 cur = conn.cursor()
6
7 cur.execute("CREATE TABLE Users (id INTEGER PRIMARY KEY, name TEXT)")
8 cur.execute("CREATE TABLE Products (id INTEGER PRIMARY KEY, name TEXT, price REAL)")
9 cur.execute("CREATE TABLE Orders (id INTEGER PRIMARY KEY, user_id INTEGER, date TEXT)")
10 cur.execute("""CREATE TABLE OrderDetails (
11     id INTEGER PRIMARY KEY,
12     order_id INTEGER,
13     product_id INTEGER,
14     quantity INTEGER,
15     FOREIGN KEY(order_id) REFERENCES Orders(id),
16     FOREIGN KEY(product_id) REFERENCES Products(id)""")
17
18 # Orders by user
19 cur.execute("SELECT * FROM Orders WHERE user_id = 1")
20
21 # Most purchased product
22 cur.execute("""SELECT product_id, SUM(quantity) as total
23     FROM OrderDetails GROUP BY product_id
24     ORDER BY total DESC LIMIT 1""")
25
26 # Revenue in month
27 cur.execute("""SELECT SUM(Products.price * OrderDetails.quantity)
28     FROM OrderDetails
29     JOIN Products ON Products.id = OrderDetails.product_id""")
30
31 print(cur.fetchall())
32
33 conn.commit()
34 conn.close()
35

```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE TERMINAL

[(None,)]

Justification

- Separates orders and details for normalization
- Improves scalability for large transactions
- Aggregation helps revenue analytics
- Optimized joins improve query performance

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

- Tables: Students, Subjects, Exams, Results.
- Define primary keys, foreign keys, and referential integrity constraints.
- Generate queries:

o Insert examination results for a student. o

Retrieve subject-wise marks for a student. o

Calculate average marks for each subject.

Expected Output:

- Normalized schema and SQL queries involving aggregation and joins.

PROMPT:

Generate university exam DB schema with Students, Subjects, Exams, Results and queries for inserting results, subject-wise marks, and average mark import sqlite3

CODE:

```
conn = sqlite3.connect("exam.db") cur
= conn.cursor()
```

```
cur.execute("CREATE TABLE Students (id INTEGER PRIMARY KEY, name TEXT)") cur.execute("CREATE
TABLE Subjects (id INTEGER PRIMARY KEY, subject_name TEXT)") cur.execute("CREATE TABLE
Exams (id INTEGER PRIMARY KEY, subject_id INTEGER)") cur.execute("""CREATE TABLE Results (
    id INTEGER PRIMARY KEY,
    student_id INTEGER,
    subject_id INTEGER,        marks
INTEGER,
    FOREIGN KEY(student_id) REFERENCES Students(id),
    FOREIGN KEY(subject_id) REFERENCES Subjects(id))""")
```

```
# Insert result cur.execute("INSERT INTO Results (student_id, subject_id, marks)
VALUES (1,1,85)")
```

```
# Subject-wise marks cur.execute("""SELECT
subject_name, marks FROM Results
JOIN Subjects ON Subjects.id = Results.subject_id
WHERE student_id = 1""")
```

```
# Average marks per subject cur.execute("""SELECT
subject_id, AVG(marks)
```

```
FROM Results GROUP BY subject_id''''')
```

```
print(cur.fetchall())
```

```
conn.commit() conn.close()
```

```
1  # Generate university exam DB schema with Students, Subjects, Exams, Results and queries for inserting results, subject-wise
2  import sqlite3
3
4  conn = sqlite3.connect("exam.db")
5  cur = conn.cursor()
6
7  cur.execute("CREATE TABLE Students (id INTEGER PRIMARY KEY, name TEXT)")
8  cur.execute("CREATE TABLE Subjects (id INTEGER PRIMARY KEY, subject_name TEXT)")
9  cur.execute("CREATE TABLE Exams (id INTEGER PRIMARY KEY, subject_id INTEGER)")
10 cur.execute("""CREATE TABLE Results (
11                 id INTEGER PRIMARY KEY,
12                 student_id INTEGER,
13                 subject_id INTEGER,
14                 marks INTEGER,
15                 FOREIGN KEY(student_id) REFERENCES Students(id),
16                 FOREIGN KEY(subject_id) REFERENCES Subjects(id)""")
17
18 # Insert result
19 cur.execute("INSERT INTO Results (student_id, subject_id, marks) VALUES (1,1,85)")
20
21 # Subject-wise marks
22 cur.execute("""SELECT subject_name, marks FROM Results
23               JOIN Subjects ON Subjects.id = Results.subject_id
24               WHERE student_id = 1""")
25
26 # Average marks per subject
27 cur.execute("""SELECT subject_id, AVG(marks)
28               FROM Results GROUP BY subject_id""")
29
30 print(cur.fetchall())
31
32 conn.commit()
33 conn.close()
34
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE

TERMINAL

• [(1, 85.0)]

Justification

- Maintains referential integrity across tables
- Supports detailed student performance tracking
- Aggregation enables academic insights
- Normalized schema avoids redundancy